IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received xx Month 2026, accepted xx Month 2026, date of publication xx Month 2026, date of current version xx Month 2026.

Digital Object Identifier 10.1109/ACCESS.2026.XXXXXXX

Mitigating Evasive Techniques Against Dynamic Binary Instrumentation with Modular Countermeasures

FRANCISCO S. S. NETO[1], MARGEFSON M. BARROS[1], and EDUARDO L. FEITOSA[1]
[1]Institute of Computing (IComp), Federal University of Amazonas (UFAM), Manaus, Amazonas 69077-000, Brazil

Corresponding author: Francisco S. S. Neto (e-mail: franciscneto@icomp.ufam.edu.br).

This work was supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES-PROEX) under Finance Code 001, and in part by the Fundação de Amparo à Pesquisa do Estado do Amazonas (FAPEAM) through the POSGRAD 2024/2025 Program.

ABSTRACT Dynamic Binary Instrumentation (DBI) enables detailed runtime observation but introduces artifacts that may reveal the analysis environment to evasive programs. This article presents TOMWare.M, a modular defensive *Pintool* built on Intel Pin for Windows to mitigate detection based on debugging indicators, process and module enumeration, environment variables, memory signatures, and timing deviations. Five controlled detector applications first served as functional oracles. Operational outcomes and changes in execution duration and resource utilization were then evaluated using 100 Windows PE x64 executables, comprising 50 benign applications and 50 malware samples. Nine conditions, including native execution, an empty tool, disabled modules, individual activation, and combined activation, were evaluated with ten repetitions per executable, yielding 9,000 recorded attempts. The environment-variable, memory-comparison, and process-enumeration countermeasures suppressed their targeted indicators through the monitored interfaces. In the timing experiment, with 30 observations per condition and all attempts retained, SkewMask increased the proportion of scores below the calibrated threshold from 36.7% to 100% while maintaining the same artificial workload. Under matched simulated-debugging conditions, `AntiDebug` suppressed the monitored process environment block (PEB) indicators in all three treatment repetitions, whereas all three baseline repetitions were positive. Among persistent executables, SkewMask increased the proportion remaining observable throughout the 60-second window from 9.6% to 52.8% for benign applications and from 32.8% to 57.6% for malware, relative to the disabled-module condition. Changes in execution duration and resource utilization are interpreted alongside possible changes in execution state. These findings support targeted indicator suppression within the evaluated interfaces and corpus, without establishing behavioral equivalence or universal instrumentation transparency.

INDEX TERMS Anti-instrumentation, countermeasures, dynamic binary instrumentation, execution transparency, Intel Pin, malware analysis.

I. INTRODUCTION

DYNAMIC program analysis complements static analysis by exposing runtime behavior such as code reconstruction, payload decryption, dynamic resolution of application programming interfaces (APIs), and environment-dependent execution paths. Dynamic binary instrumentation (DBI) inserts monitoring routines into compiled programs during execution without requiring source code, recompilation, or permanent modification of the executable. Frameworks such as Intel Pin provide fine-grained access to registers, control transfers, function calls, and memory operations [1].

Instrumentation also introduces observable changes to program execution. Translated code, additional memory regions, libraries, auxiliary processes, and timing deviations may reveal the analysis environment. Evasive programs can respond by terminating, delaying execution, following alternative paths, or suppressing their primary behavior, thereby limiting the evidence collected through dynamic analysis [2, 3].

Empirical studies illustrate the relevance of these checks. Polino et al. identified at least one anti-instrumentation technique in 15.6% of 7,006 malware samples [4]. Kim et al. examined 763,985 Windows malware samples from 2017–2020

and reported that 16.21% employed anti-analysis techniques on average [5]. These figures concern different populations and detection categories. In particular, the broader anti-analysis scope of Kim et al. should not be interpreted as a direct estimate of DBI-detection prevalence.

The literature also characterizes the range of inspection mechanisms available to evasive programs. Santos Filho et al. reviewed DBI evasion techniques and their countermeasures [3]. Park and Park examined Windows detection mechanisms involving timing, code-cache artifacts, memory signatures, just-in-time compiler behavior, and environmental artifacts [6]. Lee et al. demonstrated bypasses for anti-VM and anti-DBI checks implemented by commercial software protectors [7].

Previous work establishes that selected instrumentation checks can be identified and bypassed [4, 7, 8]. Evaluating a countermeasure nevertheless requires distinguishing suppression of an indicator from its operational effects. A check may be masked while the program changes its execution path or resource use. Combined activation further complicates attribution to individual mechanisms. Selectively enabled countermeasures allow these properties to be examined separately, while a combined configuration permits observation of their joint effects.

This study investigates five independently selectable countermeasures for Windows executables instrumented with Intel Pin. They are implemented in TOMWare.M[1], a modular defensive Pintool. `AntiDebug` clears selected debugger-related fields in the process environment block (PEB), and `ProcessEnum` filters configured process and module names through intercepted interfaces. `SanitizePinEnvVars` renames selected environment-variable entries, while `InstMemcmpMask` masks configured signatures during intercepted memory comparisons. `SkewMask` adjusts selected timing values using the excess duration measured during monitored waits.

Coverage is limited to the structures, APIs, routines, signatures, and timing sources explicitly handled by the implementation. The evaluation therefore treats indicator suppression, recorded operational outcomes, resource measurements, and agreement in observable outputs as distinct properties, without assuming universal transparency or behavioral equivalence.

The evaluation was guided by the following research questions:

- **RQ1:** To what extent does each countermeasure suppress the indicator exercised by its controlled detector, and what functional limitations remain?
- **RQ2:** What changes in recorded operational outcomes, observed duration, and resource utilization accompany individual or combined countermeasure activation in benign and malicious executables?

Functional validation used five controlled detector applications. The debugging comparison retained the same simulated stimulus in both conditions, with three runs per condition. The timing experiment fixed 30 attempts in each of three conditions and retained all 90 outcomes, with the artificial workload held constant in the main comparison. These tests assess the exercised indicators rather than the presence of evasive checks in the corpus binaries.

The operational study comprised 100 Windows Portable Executable (PE) binaries for x64: 50 benign applications and 50 malicious samples, with 25 short-lived and 25 persistent executables per class. Nine conditions covered native execution, Pin with an empty Pintool, disabled countermeasures, five individually activated countermeasures, and combined activation. Ten attempts per executable and condition yielded 9,000 recorded attempts, with condition order randomized within blocks defined by executable and repetition. Short-lived binaries had a 120-s timeout, and persistent binaries had a 60-s observation window. Outcomes were reported separately, including terminations with unavailable exit codes. Eligible duration and peak-memory contrasts were summarized using median differences within binaries and bootstrap 95% confidence intervals.

Separate diagnostic campaigns examined module activity and observable outputs. An exploratory external-observer comparison reported agreement in recorded exit status and standard-output hashes in 22 of 30 pairs between the disabled-module and combined configurations. These diagnostic observations were separate from the 9,000-attempt corpus campaign and provide limited output comparisons, without establishing equivalent control flow, API activity, or external effects.

The main contributions of this work are as follows:

- An implementation of five independently selectable countermeasures for DBI-detection surfaces on Windows, with explicit interception coverage, provided through TOMWare.M.
- A functional evaluation protocol with matched debugging stimuli and a fixed-size timing experiment that retains every outcome, separating indicator suppression from operational and behavioral claims.
- An evaluation of 100 binaries across nine conditions and ten attempts per executable and condition, totaling 9,000 recorded attempts, with paired resource contrasts and uncertainty estimates. Separate diagnostics examine module activity and limited agreement in observable outputs.
- Experimental artifacts comprising source code, controlled detectors, execution scripts, sample metadata and SHA-256 identifiers, and individual CSV and JSON records and consolidated reports for auditing the evaluation.

Section II reviews DBI and related work, and Section III describes the detection surfaces. Sections IV and V present the architecture and countermeasure implementations. Section VI reports the experimental protocol and results, Section VII discusses their interpretation and limitations, and Section VIII concludes the article.

[1] Project repository: https://github.com/TOMWare-analises/TOMWare

II. BACKGROUND AND RELATED WORK

A. DYNAMIC BINARY INSTRUMENTATION

DBI enables compiled programs to be monitored and modified during execution without requiring source code, recompilation, or permanent changes to the executable. Applications include instruction tracing, memory inspection, unpacking, and behavioral malware analysis [1, 9].

Translation-based frameworks, including Intel Pin and DynamoRIO, execute translated application code under runtime control. In Pin's just-in-time (JIT) mode, application instructions are translated and instrumented before execution from the code cache. Instrumentation routines select where analysis code is inserted; analysis routines execute when the corresponding events occur [1].

The code cache, runtime structures, auxiliary components, and analysis routines create differences from native execution. Their visibility depends on the framework, architecture, version, and instrumentation configuration, so a check that detects one environment need not detect another [6, 10].

B. TRANSPARENCY AND OBSERVABILITY

Transparency concerns preservation of the properties observable by the application. Bruening et al. described code, data, and concurrency transparency and discussed the difficulty of concealing timing effects [11]. Even a functionally correct execution may remain detectable through memory contents, code pointers, operating-system interactions, exceptions, or timing measurements [9, 10].

We assess indicator suppression through explicitly exercised checks: a positive baseline indicator must cease to be reported through the monitored interface under a matched configuration. For timing, the evidence consists of score distributions and proportions below a calibrated threshold across a fixed number of attempts, retaining every outcome. These observations do not demonstrate that equivalent checks through other interfaces are also suppressed.

Operational outcomes distinguish launch failure, termination, timeout, and observation-window attainment. Agreement in recorded exit status and output hashes provides a limited comparison of observables. Neither process survival nor output agreement establishes equivalent control flow, API activity, or external effects. Resource contrasts are also interpreted separately because different execution paths or application progress can change CPU and memory measurements.

C. ANTI-INSTRUMENTATION AND OBSERVABLE SURFACES

Anti-instrumentation techniques inspect artifacts or execution differences that may reveal DBI. D'Elia et al. characterized surfaces involving timing, code pointers, memory contents and permissions, runtime structures, operating-system interactions, and translation behavior [9]. Santos Filho et al. distinguished direct artifact inspection from indirect checks of functional or resource-related limitations [3]. Park and Park examined Windows techniques involving timing, code-cache artifacts, memory signatures, JIT behavior, and environmental artifacts [6].

The five surfaces examined here form a subset of these taxonomies. Selection was guided by observable Intel Pin indicators and the feasibility of implementing countermeasures at the Pintool level. The interception mechanisms and their boundaries are described in Sections III and V.

D. RELATED WORK

Artifact detection has been studied through controlled tests and framework-level analysis. Falcón and Riva introduced eXait, a collection of anti-instrumentation checks focused on Intel Pin [12]. Kirsch et al. investigated weaknesses exposed by Pin's execution model, illustrating the risks of placing an analysis runtime within an adversarial program's address space [13]. Palmaro and Franchina demonstrated detection through low-level memory scanning [14], while D'Elia et al. evaluated conspicuous features across DBI systems [10].

The execution point of a check is also relevant. Padhy et al. combined machine-bound execution through digital rights management with anti-debugging in thread-local storage (TLS) callbacks before the main entry point [15]. Their work motivates examining initialization order, but does not establish that the countermeasures evaluated here precede every early check.

Defensive systems address these surfaces at different architectural levels. Rodríguez et al. introduced PinVMShield, a Windows-focused Pintool with a plugin architecture for addressing analysis-aware malware [2]. Santos Filho et al. extended it with API-based concealment of Pin-reserved TLS slots, filtering of code-cache-region queries, and detection of execution attempts from nonexecutable memory [16]. The latter mechanism detects the execution attempt and does not establish complete restoration of native memory-protection behavior.

Polino et al. implemented Arancino on Intel Pin and demonstrated countermeasures, corpus-level measurement, and evasion-resilient unpacking [4]. In the cited preprint, Crăciun et al. introduced COBAI, a dedicated DBI engine with cooperating plugins for handling engine and operating-system artifacts, together with an aggregated transparency test suite [17]. Gaber et al. developed Peekaboo to extract instruction-level behavior and network activity from evasive Windows malware [18].

Protector-specific studies provide complementary evidence. Park et al. used Intel Pin to detect and bypass anti-debugging checks in Windows protectors [8]. Lee et al. examined checks for virtual machines and DBI in five commercial protectors and evaluated bypass procedures using 1,573 test executables [7].

Measurement studies distinguish the occurrence of a monitored operation from its use in evasion. Kim et al. developed EvDetector to identify anti-analysis checks and determine whether their results influence execution paths [5]. Consequently, module initialization or an intercepted API call alone does not demonstrate an anti-instrumentation decision or recovered malware behavior. Norby et al. measured anti-debugging techniques on Windows and Linux, including sta-

tistical characterization of their execution cost [19]. Their measurements concern detection checks themselves, whereas this study examines recorded effects of enabling countermeasures.

Survey evidence further motivates separating coverage from operational evaluation. Santos Filho et al. documented 26 evasion techniques, with countermeasures for 12 at the time of their review, and identified gaps in practical effectiveness and performance evaluation [3].

Architectural alternatives change where or how execution is observed. Zeng et al. introduced PEMU, a Pin-compatible framework supporting introspection of user processes and operating-system kernels from outside the virtual machine [20]. Zhao et al. developed LibIHT, which collects hardware branch records to reconstruct control flow [21]. Its branch-address observations provide different information from full software instrumentation, trading richer execution semantics for lower interference in the evaluated setting. These approaches therefore differ from countermeasures implemented within an existing Pintool.

Within this landscape, the present study evaluates five selected countermeasures through controlled detectors, individual and combined activation, process outcomes, resource contrasts, and limited comparisons of externally recorded outputs. Table 1 summarizes the architecture and objectives of closely related systems. Selective activation is not claimed to be unique to TOMWare.M, and the qualitative comparison does not establish superior coverage, transparency, behavioral fidelity, or performance.

III. EVASIVE TECHNIQUES AGAINST DYNAMIC BINARY INSTRUMENTATION

This section examines five literature-grounded detection surfaces addressed by the selected countermeasures. Table 2 summarizes representative indicators and inspection mechanisms. The listed mechanisms describe detection possibilities and do not imply that all are intercepted by TOMWare.M. Implementation details and interface-specific coverage are provided in Section V.

A. INSPECTION OF DEBUGGING INDICATORS

Anti-debugging techniques query states maintained by the operating system or exploit behavioral differences to determine whether a process is being observed by a debugger. Common Windows checks include `IsDebuggerPresent` and `CheckRemoteDebuggerPresent`, as well as direct inspection of the `BeingDebugged` and `NtGlobalFlag` fields of the *Process Environment Block* (PEB) [8, 19].

The `NtQueryInformationProcess` function exposes an additional detection surface through information classes such as `ProcessDebugPort`, `ProcessDebugObjectHandle`, and `ProcessDebugFlags`. Interpretation depends on the requested class. For `ProcessDebugFlags`, a zero output value following a successful query may indicate debugging. The NTSTATUS return value reports query success or failure and must be distinguished from the process information written to the output buffer [6, 8].

These checks are not specific to DBI: Intel Pin is not a conventional debugger, and debugger-related queries need not expose instrumentation [7, 9]. The evaluated `AntiDebug` module clears two PEB fields, `BeingDebugged` and `NtGlobalFlag`. Clearing the former may indirectly affect `IsDebuggerPresent`, but the module does not directly replace debugger-query APIs.

B. PROCESS AND MODULE ENUMERATION

Loader processes and auxiliary libraries can expose an instrumentation environment through their names, paths, or relationships. Names such as `pin.exe` or `drrun.exe` may be visible depending on the framework, version, and launch mode [6, 12]. A name alone is not conclusive because unrelated programs can use the same identifier.

On Windows, Tool Help enumeration begins with `CreateToolhelp32Snapshot`. Process entries are subsequently obtained through `Process32First` and `Process32Next`. Module entries are retrieved through `Module32First` and `Module32Next`. Correct filtering must preserve unrelated entries and the expected end-of-enumeration behavior.

Module presence can also be tested through the ANSI and Unicode variants of `GetModuleHandle`. For system-wide process inspection, `NtQuerySystemInformation` with the `SystemProcessInformation` class exposes process information, including process and parent identifiers.

The evaluated `ProcessEnum` module filters configured base names through the `Process32*` and `Module32*` families and through `GetModuleHandleA` and `GetModuleHandleW`. This changes query results without removing processes or unloading modules. Native queries through `NtQuerySystemInformation`, alternative APIs, and direct inspection of internal structures remain outside its coverage.

C. ENVIRONMENT-VARIABLE ARTIFACTS

Instrumentation loaders and supporting components may create or propagate environment variables during initialization. A program can inspect these entries through ANSI or Unicode environment-query APIs, C Runtime Library (CRT) routines, or direct access to the environment block referenced by the PEB [3, 4].

The exposed entries depend on the framework version and configuration. A prefix such as `PIN_` is therefore an environment-specific signature, and legitimate applications may use similar names. Direct access and runtime-maintained copies also limit the coverage of API-only interception.

The evaluated `SanitizePinEnvVars` module renames the `PIN_` prefix to `ZZZ_` in writable entries of the PEB-referenced environment block at application startup, preserving entry lengths. Subsequent queries of that block no longer locate the modified entries under their original names. This effect extends to the CRT when its internal environment repre-

TABLE 1. Reported architecture and objectives of closely related systems.

System	Instrumentation basis	Primary objective and evaluation focus
PinVMShield [2, 16]	Windows-focused Intel Pin Pintool with extensible plugins.	Mitigate analysis-environment checks. The cited extension addresses Pin-reserved TLS slots, code-cache queries, and detection of execution from nonexecutable memory.
Arancino [4]	Intel Pin-based defensive Pintool.	Suppress anti-instrumentation checks and support evasion-resilient unpacking and empirical measurement.
COBAI [17]	Dedicated DBI engine with cooperating plugins.	Address instrumentation-engine and operating-system artifacts, with an aggregated transparency test suite.
Peekaboo [18]	DBI-based malware-analysis environment.	Extract instruction-level behavior and network activity from evasive Windows malware.
TOMWare.M	Intel Pin-based Pintool with five selectable countermeasures.	Evaluate selected indicator suppression and compare individual and combined activation using recorded process outcomes and resource measurements.

Descriptions summarize the cited systems; COBAI is described from the cited preprint. No common experimental benchmark of these systems was conducted in this study.

TABLE 2. Representative DBI detection surfaces and inspection mechanisms reported in the literature.

Surface	Observable indicator	Representative inspection mechanisms	References
Debugging	Debugger-related process state and query results.	PEB fields; `IsDebuggerPresent`; `CheckRemoteDebuggerPresent`; `NtQueryInformationProcess`.	[6, 7, 8, 19]
Processes and modules	Names and relationships of loader processes and auxiliary libraries.	`CreateToolhelp32Snapshot`; `Process32First` and `Process32Next`; `Module32First` and `Module32Next`; `GetModuleHandle`; `NtQuerySystemInformation`.	[4, 6, 12]
Environment	Variable names or values associated with the instrumentation environment.	Environment-query APIs; runtime-library queries; direct inspection of the environment block referenced by the PEB.	[3, 4, 6]
Memory	Runtime signatures and characteristic memory-region properties.	Direct scans; `VirtualQuery` and `VirtualQueryEx`; comparison routines such as `memcmp` and `wmemcmp`.	[9, 10, 14]
Timing	Elapsed-time deviations and differences between initial and cached execution.	`QueryPerformanceCounter`; `GetTickCount`; `RDTSC` and `RDTSCP`; elapsed-time checks around waits or repeated execution.	[6, 11, 22]

sentation is constructed from the sanitized block. The entries remain present under their new names; residual copies and variables introduced after sanitization remain outside this operation.

D. MEMORY SIGNATURES AND REGIONS

Execution under DBI may introduce memory regions containing translated code, internal data structures, auxiliary libraries, and character strings associated with the instrumentation mechanism. In Intel Pin, names such as `pin.exe`, `pinvm.dll`, and `pinipc.dll` may remain accessible within the process address space, depending on the version and configuration being used.

Detection routines can use `VirtualQuery` or `VirtualQueryEx` to inspect memory-region attributes and then scan accessible contents through library comparisons or custom loops. Examples include `memcmp`, `wmemcmp`, and `RtlCompareMemory`. Palmaro and Franchina demonstrated detection through low-level memory scanning [14].

The `InstMemcmpMask` module operates on an explicit set of comparison routines and on signatures defined internally or loaded from a configuration file. It returns a non-matching result according to the intercepted routine's conventions when the second operand matches a monitored signature and the comparison length matches the expected length. The buffers remain unchanged. Coverage therefore depends on the routine, operand position, signature, and length; custom comparison loops, reversed operands, partial searches, and memory-page inspections remain outside this mechanism.

E. TIMING ANOMALIES AND DEVIATIONS

Dynamic translation, code-cache generation, and analysis routines add work to application execution. Programs may observe these delays through timing APIs, including

QueryPerformanceCounter, GetTickCount, and GetTickCount64, or through direct RDTSC and RDTSCP reads [11, 22].

A timing check computes an interval, $\Delta t = t_2 - t_1$, and compares it with an environment-specific threshold. Another strategy compares an initial execution that requires JIT translation with subsequent executions from the code cache [6]. Scheduling, interrupts, virtualization, power management, and concurrent load can also affect the measured interval, limiting the specificity of threshold-based detection.

The SkewMask module intercepts selected suspension functions and timing sources. For monitored relative waits, it accumulates the nonnegative excess of the observed interval over the requested delay and subtracts the corresponding offset from intercepted clock values. Direct RDTSC and RDTSCP reads, absolute delays, unmonitored clocks, and external counters remain outside its coverage.

Timing compensation also requires a coherent timeline across clock sources and threads. If overlapping waits contribute to a shared offset, summing their excesses can count a shared elapsed interval more than once. Consistency must therefore be assessed through sequential and concurrent waits, repeated clock reads, and alertable waits that return early, including checks for backward movement of clocks expected to be monotonic. Suppression of a threshold-based oracle result alone does not establish these properties. The timing protocol and its validation limits are reported in Section VI.

IV. TOMWARE.M ARCHITECTURE

The five countermeasures are implemented in TOMWare.M, a modular *Pintool* built on Intel Pin [1]. Figure 1 shows the main components and activation flow. The core component, *TOMWare.M Main*, parses command-line options, initializes the tool, registers the callbacks required by the selected countermeasures, and starts the target program under Pin.

The -dd, -dp, -de, -dm, and -do parameters enable the AntiDebug, ProcessEnum, SanitizePinEnvVars, InstMemcmpMask, and SkewMask modules, respectively. The -da parameter enables all five countermeasures simultaneously. The disabled-module reference loads TOMWare.M without enabling any countermeasure. The corpus campaign compares this reference with individual and combined activation.

The installation mechanism depends on the detection surface. Countermeasures that modify process structures register routines for application startup or main-image loading. API-based countermeasures register image-loading callbacks that replace selected routines through Pin's interfaces. The InstMemcmpMask module locates comparison routines through available image symbols and installs wrappers designed to preserve calling conventions and apply routine-specific return conventions when masking a matching signature. The structures and interfaces addressed by each module are detailed in Section V.

Experimental orchestration is implemented through external scripts. For the corpus campaign, these scripts randomize condition order within each block defined by an executable and a repetition. They enforce a 120-s timeout for short-lived executables and a 60-s observation window for persistent executables, collect observed duration and resource measurements, and generate CSV and JSON records and consolidated reports. Run scheduling, observation limits, and measurement collection therefore belong to the experimental infrastructure rather than to the countermeasure implementations.

V. COUNTERMEASURE IMPLEMENTATION

The five countermeasures target selected Intel Pin indicators by modifying process structures or intercepting specific APIs and library routines. The diversity of DBI detection surfaces documented in prior work motivates the use of different countermeasure mechanisms [3, 6, 9, 10]. This section describes each implementation, its coverage and limitations, and the interfaces exercised by the functional tests.

A. ANTIDEBUG

Anti-debugging techniques can inspect debugging state through Windows APIs, process information classes, and fields maintained in the Process Environment Block (PEB). Common checks include IsDebuggerPresent, CheckRemoteDebuggerPresent, and queries through NtQueryInformationProcess. Direct PEB inspection typically examines the BeingDebugged and NtGlobalFlag fields [6, 8, 19].

Broader debugger-concealment approaches address different detection mechanisms through changes to execution state, as in Apate, or through hardware-assisted isolation, as proposed by Zhang et al. [23, 24]. The AntiDebug module implements a targeted user-space countermeasure for two PEB fields.

The module obtains the PEB address through the Thread Environment Block (TEB) and writes zero to the BeingDebugged field. It then locates NtGlobalFlag using an architecture-dependent offset. After verifying that the destination address is writable, it also sets this field to zero. Sanitization is registered for application startup and is additionally performed when the executable's main image is loaded.

Because IsDebuggerPresent derives its result from BeingDebugged, clearing this field may indirectly change the value returned by the API. The evaluated version does not directly replace IsDebuggerPresent, CheckRemoteDebuggerPresent, or NtQueryInformationProcess. Its direct coverage is restricted to the two PEB fields; responses obtained independently of these fields remain outside this coverage.

1) Controlled Functional Validation

The TestAntiDebug.exe oracle was executed three times per condition, using -gdb -q for the baseline and -gdb -dd -q with the countermeasure. The -gdb option

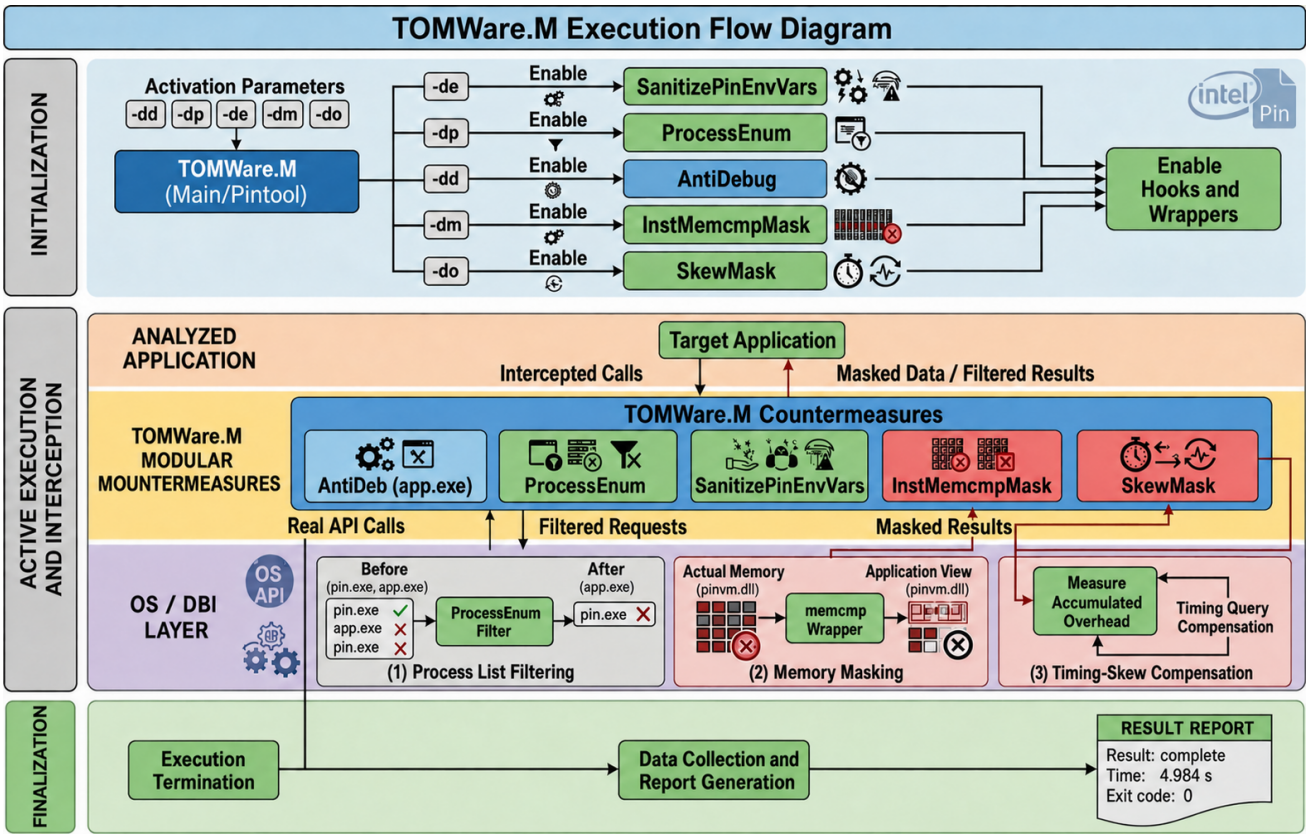


FIGURE 1. Modular countermeasure architecture and activation flow in TOMWare.M.

artificially introduced debugging indicators into the PEB in both conditions; enabling `-dd` was the only difference between the configurations. The routine that introduced the indicators was registered before the sanitization routine. No debugger was attached to the process in this test.

Table 3 shows positive indicators in all three baseline executions and none in the three executions with the countermeasure. The result supports suppression of the targeted checks under the controlled stimulus, without establishing effectiveness against an attached debugger or other detection mechanisms.

In a representative pair of outputs, `CheckRemoteDebuggerPresent`, `ProcessDebugPort` and `ProcessDebugObjectHandle` already produced negative results in the baseline and remained negative under treatment. This example does not provide a positive control for suppression through these additional interfaces.

2) Operational Outcomes in the Corpus

The evaluation used 100 Windows PE executables, comprising 50 benign and 50 malicious samples. Each execution profile included 25 benign and 25 malicious executables. Ten attempts per binary and condition yielded 1,000 baseline attempts and 1,000 with `AntiDebug`. The comparison used `-q` and `-dd -q`, with randomized order within blocks defined by binary and repetition, without the stimulus introduced by `-gdb`.

The collector did not record the exit code in 777 baseline attempts and 783 attempts with the countermeasure. These records do not distinguish normal termination from failure. All these attempts remain in Table 4, without being classified as successful completions or confirmed failures.

Among persistent executables, the difference in observation-window attainment was -1.4 percentage points, with a 95% confidence interval (CI) of [-3.8, +1.0]. This exploratory interval was computed using a paired bootstrap over the 50 binaries, stratified by class, with 2,000 resamples and seed 705851366. The interval includes zero and does not establish equivalence between conditions. Process survival also does not demonstrate that the same tasks were performed.

3) Observed Duration and Resource Utilization

Table 5 presents the medians of the differences computed within each binary from the medians recorded in each condition. The 95% CIs were computed using the percentile method, with 2,000 paired resamples of binaries, seed 705851366, and reinitialization of the generator for each stratum and metric.

The filters for this reanalysis were applied post hoc: finite, positive duration in both conditions; finite memory values and at least one resource observation per condition. A total of 98 pairs were retained. Executables `MAL_33` and `MAL_37` had

TABLE 3. Functional validation of AntiDebug under the same controlled debugging stimulus in both conditions, with three oracle executions per configuration.

Configuration	n	PEB. BeingDebugged	PEB. NtGlobalFlag	IsDebuggerPresent
Baseline `-gdb -q`	3	3/3	3/3	3/3
AntiDebug enabled `-gdb -dd -q`	3	0/3	0/3	0/3

n denotes separate process executions. Cells report executions with a positive indicator divided by n, rather than raw field values. The three checks use the same executions; the API provides a reading associated with the `BeingDebugged` field.

TABLE 4. Recorded AntiDebug outcomes by execution profile. Each profile contains 50 executables and 500 attempts per condition.

Recorded outcome	Modules disabled `-q`	AntiDebug enabled `-dd -q`	Difference (pp)
(a) Short-lived executables			
Termination with unavailable exit code	406 (81.2%)	406 (81.2%)	0.0
120-s timeout	88 (17.6%)	89 (17.8%)	+0.2
Launch failure	6 (1.2%)	5 (1.0%)	−0.2
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**
(b) Persistent executables			
Termination with unavailable exit code	371 (74.2%)	377 (75.4%)	+1.2
Observation-window attainment at 60 s	106 (21.2%)	99 (19.8%)	−1.4
Launch failure	23 (4.6%)	24 (4.8%)	+0.2
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**

Cells report counts (percentages of 500 attempts). Difference: the AntiDebug percentage minus the baseline percentage, in percentage points (pp). All attempts are included.

zero recorded duration and no resource measurements in either condition; they were excluded only from these contrasts, while their attempts remained in the operational-outcome table.

In the malicious short-lived stratum, the median duration difference was +148.9 ms, with a 95% CI of [24.7, 842.0] ms. The duration intervals for the other strata and all peak-memory intervals included zero. These results are exploratory and do not demonstrate behavioral equivalence or absence of cost.

CPU-time summaries had a median of zero in both conditions across all strata. However, 19 of the 98 binaries with available records had a positive median CPU time in the baseline, compared with 20 with `AntiDebug`. In addition, 21 pairs had nonzero CPU-time differences. The supplied summaries do not allow the origin of the numerous zero values to be verified; therefore, this metric does not support a conclusive estimate of CPU cost.

Internal counters remained disabled in the corpus campaign, and queries to the PEB fields were not verified. The operational and resource results do not establish which executables performed the targeted checks or whether enabling the countermeasure allowed additional malicious activity to be observed. Suppression of the PEB indicators was observed separately in the controlled functional test.

B. PROCESSENUM

Process and module enumeration can reveal loaders, auxiliary libraries, and other components associated with the instrumentation environment [4, 6, 12]. On Windows, these inspections may use the Tool Help API family, module-query functions, or native system-information interfaces.

The `ProcessEnum` countermeasure, enabled by `-dp`, replaces `Process32First`, `Process32Next`, `Process32FirstW`, and `Process32NextW`. When an entry's base name matches `pin.exe` through a case-insensitive comparison, the module advances the enumeration until it finds an allowed entry or reaches the end.

The same filtering procedure is applied to `Module32First`, `Module32Next`, `Module32FirstW`, and `Module32NextW`. The monitored base names are `pinvm.dll`, `pinipc.dll`, and `TOMWare.dll`, also compared without regard to case.

The `GetModuleHandleA` and `GetModuleHandleW` functions are also replaced. When the supplied name or path has a base name matching one of the monitored modules, the replacement returns a null handle. Other queries are forwarded to the original function.

Filtering modifies the information returned by the intercepted interfaces; it does not remove processes or modules from the execution environment.

TABLE 5. Paired differences in observed duration and peak memory with AntiDebug. Estimates are medians of per-binary differences, enabled condition minus baseline; 95% CIs are shown in brackets.

Class and profile	n	Duration difference (ms)	Peak-memory difference (MB)
Benign, short-lived	25	−36.7 [−312.8, 107.1]	−0.010 [−0.561, 0.639]
Benign, persistent	25	+51.1 [−49.9, 354.0]	−0.008 [−0.533, 0.670]
Malicious, short-lived	25	+148.9 [24.7, 842.0]	+0.174 [−0.002, 0.713]
Malicious, persistent	23	+15.8 [−573.5, 289.0]	−0.406 [−1.629, 0.482]

n denotes paired binaries with available measurements. MB retains the unit identified in the supplied records. The contrasts describe the observed executions without verification that equivalent work was completed.

1) Controlled Functional Validation

The `TestProcessEnum.exe` oracle used Unicode process enumeration and found three entries named `pin.exe` in the baseline and none with `-dp` or `-da`. Table 6 reports one execution per configuration; the three baseline entries do not represent three independent repetitions.

In the complementary `TestModule32.exe` test, enumeration terminated and `kernel32.dll` remained available in all three recorded configurations. Native execution enumerated 31 modules, compared with 29 in both the baseline and the run with `-dp`. The test's artifact indicator was already negative in the baseline. These records verify enumeration termination and the availability of an unfiltered entry, but do not demonstrate concealment of a monitored module.

The suppression observed in the process test remains restricted to the query exercised by the oracle. The presented records do not include specific positive controls for module concealment through `Module32*` or `GetModuleHandleA/W`. The result with `-da` is also limited to the tested process indicator and does not establish combined validation of all interfaces across the five modules.

2) Operational Outcomes in the Corpus

The `ProcessEnum` corpus evaluation comprised 100 executables, with ten attempts per condition and executable, totaling 1,000 baseline attempts and 1,000 with the countermeasure. Each execution profile included 25 benign and 25 malicious executables. Table 7 separates the profiles because the 120-s timeout and the 60-s observation window represent different criteria.

The collector did not record the exit code in 777 baseline attempts and 785 attempts with the countermeasure. In these cases, the data do not distinguish normal termination from failure. These attempts remain included, without being classified as successful completions or confirmed failures.

Among persistent executables, the difference in observation-window attainment was -1.6 percentage points, with a 95% confidence interval (CI) of [-3.6, +0.2]. This exploratory interval was computed using a paired bootstrap over the 50 binaries, stratified by class, with 2,000 resamples and seed 705851366. The interval includes zero and does not establish equivalence between conditions. Process survival also does not demonstrate that the same tasks were performed.

3) Observed Duration and Resource Utilization

Table 8 presents the medians of the differences computed within each binary from the medians recorded in each condition. The 95% CIs were computed using the percentile method, with 2,000 paired resamples of binaries, seed 705851366, and reinitialization of the generator for each stratum and metric.

The filters for this reanalysis were applied post hoc: finite, positive duration in both conditions; finite memory values and at least one resource observation per condition. A total of 98 pairs were retained. Executables `MAL_33` and `MAL_37` had zero recorded duration and no resource measurements in either condition; they were excluded only from these contrasts, while their attempts remained in the operational-outcome table.

In the two benign strata, the median duration differences corresponded to increases of approximately 3.8 and 3.9 s, with entirely positive CIs. These measurements describe the observed process durations; they do not establish the time required to complete equivalent tasks.

CPU-time summaries had a median of zero in both conditions across all strata. However, 19 of the 98 binaries with available records had a positive median CPU time in the baseline, compared with 18 with `ProcessEnum`. The supplied summaries do not allow the origin of the numerous zero values to be verified; therefore, this metric does not support a conclusive estimate of CPU cost.

The `ProcessEnum` module does not intercept `NtQuerySystemInformation`, Process Status API (PSAPI) functions, or direct inspection of internal process structures. Its coverage does not include general sanitization of process relationships, services, identifiers, or paths, nor concealment of components whose names fall outside the filtering rules.

TABLE 6. Functional test of process concealment by ProcessEnum, with one oracle execution per configuration.

Configuration	Options	Executions	`pin.exe` entries
Modules disabled	`-q`	1	3
ProcessEnum enabled	`-dp -q`	1	0
All modules enabled	`-da -q`	1	0

TABLE 7. Recorded ProcessEnum outcomes by execution profile. Each profile contains 50 executables and 500 attempts per condition.

Recorded outcome	Modules disabled `-q`	ProcessEnum enabled `-dp -q`	Difference (pp)
(a) Short-lived executables			
Termination with unavailable exit code	406 (81.2%)	405 (81.0%)	−0.2
120-s timeout	88 (17.6%)	91 (18.2%)	+0.6
Launch failure	6 (1.2%)	4 (0.8%)	−0.4
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**
(b) Persistent executables			
Termination with unavailable exit code	371 (74.2%)	380 (76.0%)	+1.8
Observation-window attainment at 60 s	106 (21.2%)	98 (19.6%)	−1.6
Launch failure	23 (4.6%)	22 (4.4%)	−0.2
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**

Cells report counts (percentages of 500 attempts). Difference: the ProcessEnum percentage minus the baseline percentage, in percentage points (pp). All attempts are included.

TABLE 8. Paired differences in observed duration and peak memory with ProcessEnum. Estimates are medians of per-binary differences, enabled condition minus baseline; 95% CIs are shown in brackets.

Class and profile	n	Duration difference (ms)	Peak-memory difference (MB)
Benign, short-lived	25	+3 843.9 [3 427.1, 4 088.4]	+0.475 [0.145, 0.975]
Benign, persistent	25	+3 906.6 [3 521.0, 4 225.7]	+0.434 [−0.855, 1.281]
Malicious, short-lived	25	+2 783.7 [52.1, 4 082.6]	+0.480 [−0.156, 0.656]
Malicious, persistent	23	+2 673.9 [−18.0, 4 063.8]	+0.219 [−0.438, 0.680]

n denotes paired binaries with available measurements. MB retains the unit identified in the supplied records. The contrasts describe the observed executions without verification that equivalent work was completed.

Internal counters remained disabled in the corpus campaign. Consequently, the operational and resource results do not establish which executables performed the intercepted queries or whether enabling the countermeasure recovered previously suppressed behavior.

C. SANITIZEPINENVVARS

Environment variables may contain identifiers introduced during initialization of the instrumentation environment. Previous studies show that variables inherited by the instrumented process may reveal the DBI loader or the underlying JIT engine [3, 4, 6, 10, 13]. Under Intel Pin, the environment may contain entries such as `PIN_CRT_TZDATA`, `PIN_APP_LD_LIBRARY_PATH`, and `PIN_VM_LD_LIBRARY_PATH`. These entries can be queried through Windows APIs, C runtime library (CRT) functions, or direct inspection of the environment block referenced by the PEB.

The `SanitizePinEnvVars` countermeasure, enabled by `-de`, registers a routine that runs during application startup, before the target program is initialized. The routine obtains the environment block through the process parameters referenced by the PEB and traverses its sequence of entries.

When a variable name begins with `PIN_`, with case-insensitive matching of the first three letters, the module checks whether the entry is writable and replaces those letters

with ZZZ. The prefix therefore changes from PIN_ to ZZZ_. This substitution preserves the entry length and block layout without removing or relocating the variable.

The countermeasure modifies the environment block without replacing query functions. In representations derived from the sanitized block, GetEnvironmentVariable does not find the original name, whereas GetEnvironmentStrings exposes the renamed entries. The effect on CRT routines, such as _dupenv_s and getenv, depends on their internal representations being constructed from the sanitized environment.

1) Controlled Functional Validation

The evaluation used one recorded execution of TestGetEnvironmentVars per condition, comparing -q with -de -q. In the baseline, the oracle identified PIN_CRT_TZDATA in the environment block and through _dupenv_s. The test summary recorded this name as present in the baseline and absent with the countermeasure, as shown in Table 9.

The positive control is provided by PIN_CRT_TZDATA, identified in the baseline but not under treatment. The other two names were already absent in the baseline; their results do not constitute positive suppression controls. The functional evidence is limited to the observed name and the queries exercised by the oracle, without individually validating all the API variants and CRT routines mentioned.

2) Operational Outcomes in the Corpus

The evaluation used 100 Windows PE executables, comprising 50 benign and 50 malicious samples. Each execution profile included 25 benign and 25 malicious executables. Ten attempts per binary and condition yielded 1,000 baseline attempts and 1,000 with sanitization enabled. The comparison used -q and -de -q, with randomized order within blocks defined by binary and repetition.

The collector did not record the exit code in 777 attempts in each condition. These records do not distinguish normal termination from failure. All these attempts remain in Table 10, without being classified as successful completions or confirmed failures.

Among persistent executables, the difference in observation-window attainment was +0.6 percentage points, with a 95% confidence interval (CI) of [-1.2, +2.6]. This exploratory interval was computed using a paired bootstrap over the 50 binaries, stratified by class, with 2,000 resamples and seed 705851366. The interval includes zero and does not establish equivalence between conditions. Process survival also does not demonstrate that the same tasks were performed.

3) Observed Duration and Resource Utilization

Table 11 presents the medians of the differences computed within each binary from the medians recorded in each condition. The 95% CIs were computed using the percentile method, with 2,000 paired resamples of binaries, seed 705851366, and reinitialization of the generator for each stratum and metric.

The filters for this reanalysis were applied post hoc: finite, positive duration in both conditions; finite CPU and memory values and at least one resource observation per condition. A total of 98 pairs were retained for duration and 97 for CPU and memory. Executables MAL_33 and MAL_37 had zero recorded duration and no resource measurements in either condition. Executable BEN_32 had no CPU or memory measurements with sanitization enabled but remained in the duration contrast. These exclusions do not alter the operational-outcome counts.

All duration and peak-memory intervals included zero. These results are exploratory and do not demonstrate absence of cost or behavioral equivalence between conditions.

CPU-time summaries had a median of zero in both conditions across all strata. However, 18 of the 97 binaries with available paired records had a positive median CPU time in the baseline, compared with 20 with sanitization enabled. In addition, 21 pairs had nonzero CPU-time differences. The supplied summaries do not allow the origin of the numerous zero values to be verified; therefore, this metric does not support a conclusive estimate of CPU cost.

The coverage of SanitizePinEnvVars is restricted to writable entries present in the environment block accessed at startup whose names begin with PIN_. Variables introduced later, copies maintained in other memory regions, identifiers matching different patterns, and other Intel Pin artifacts remain outside this scope.

Internal counters remained disabled in the corpus campaign, and the environment queries performed by each executable were not verified. The operational and resource results do not establish which samples searched for the sanitized names or whether the countermeasure allowed additional malicious activity to be observed.

D. INSTMEMCMPMASK

DBI environments may introduce identifiable data structures, memory regions, and strings into the address space of the instrumented process. Memory-scanning techniques exploit these artifacts to locate components of the instrumentation engine [6, 10, 13, 14]. In the evaluated Intel Pin configuration, the default monitored signatures are PIN_, pin.exe, pinvm.dll, and pinipc.dll. Additional signatures can be loaded from a configuration file.

The InstMemcmpMask countermeasure, enabled by -dm, examines symbols in loaded images and replaces selected comparison routines with wrappers having compatible function signatures. The implementation covers memcmp, _memcmp, __acrt_memcmp, bcmp, memicmp, _memicmp, _memicmp_l, memcmp_s, wmemcmp, wmemcmp_s, RtlCompareMemory, and RtlEqualMemory.

Before calling the original routine, the wrapper examines the second operand and the specified comparison length. When this operand exactly matches a monitored signature and

TABLE 9. Environment-variable oracle results, with one execution per condition.

Variable name	Modules disabled `-q`	Sanitization enabled `-de -q`
`PIN_CRT_TZDATA`	1	0
`PIN_APP_LD_LIBRARY_PATH`	0	0
`PIN_VM_LD_LIBRARY_PATH`	0	0

1 indicates that the name was identified in the oracle summary; 0 indicates absence. The three rows correspond to checks within the same execution of each condition, rather than independent repetitions.

TABLE 10. Recorded SanitizePinEnvVars outcomes by execution profile. Each profile contains 50 executables and 500 attempts per condition.

Recorded outcome	Modules disabled `-q`	Sanitization enabled `-de -q`	Difference (pp)
(a) Short-lived executables			
Termination with unavailable exit code	406 (81.2%)	409 (81.8%)	+0.6
120-s timeout	88 (17.6%)	90 (18.0%)	+0.4
Launch failure	6 (1.2%)	1 (0.2%)	−1.0
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**
(b) Persistent executables			
Termination with unavailable exit code	371 (74.2%)	368 (73.6%)	−0.6
Observation-window attainment at 60 s	106 (21.2%)	109 (21.8%)	+0.6
Launch failure	23 (4.6%)	23 (4.6%)	0.0
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**

Cells report counts (percentages of 500 attempts). Difference: the sanitization-enabled percentage minus the baseline percentage, in percentage points (pp). All attempts are included.

TABLE 11. Paired differences in observed duration and peak memory with SanitizePinEnvVars. Estimates are medians of per-binary differences, enabled condition minus baseline; 95% CIs are shown in brackets.

Class and profile	n_t	Duration difference (ms)	n_m	Peak-memory difference (MB)
Benign, short-lived	25	−7.1 [−179.2, 162.7]	25	−0.104 [−0.619, 0.441]
Benign, persistent	25	+8.8 [−166.2, 99.6]	24	+0.488 [−0.688, 1.129]
Malicious, short-lived	25	+6.7 [−47.9, 460.6]	25	+0.121 [−0.139, 0.504]
Malicious, persistent	23	−52.3 [−234.2, 407.7]	23	+0.188 [−0.348, 0.574]

n_t and n_m denote paired binaries with available duration and memory measurements, respectively. MB retains the unit identified in the supplied records. The contrasts describe the observed executions without verification that equivalent work was completed.

has the expected length, the handler produces a nonmatching result according to the intercepted routine's return semantics. Otherwise, the call is forwarded to the original routine. The contents of the compared buffers are not modified.

Masking affects the results of intercepted comparisons and does not physically remove signatures from memory. The parameter and return conventions of the different routines are part of the implementation scope; their functional validation must be distinguished from the existence of the corresponding replacements.

1) Controlled Functional Validation

The `TestMemoryScan.exe` oracle traverses committed, accessible memory regions and compares candidate locations against the four default signatures using `memcmp`. Table 12 reproduces the counts from the previously documented illustrative comparison: 212, 11, 14, and 18 baseline matches, respectively, and none with the configured masking.

The supplementary records for the `-q` versus `-dm -q` comparison contain strings with `PIN_` in the baseline and zero counts for `PIN_` and `pin.exe` under treatment. These excerpts do not provide a new consolidated summary of all

TABLE 12. Matches reported by TestMemoryScan.exe in the illustrative functional comparison of InstMemcmpMask.

Signature	Modules disabled	Masking enabled
`PIN_`	212	0
`pin.exe`	11	0
`pinvm.dll`	14	0
`pinipc.dll`	18	0

Baseline: `-q`. Masking: `-dm -q -sf config/signatures.txt`, as in the original illustrative comparison. Cells represent scan matches, rather than numbers of independent executions.

four signatures and were not used to replace the counts from the illustrative comparison.

The functional evidence presented is restricted to the `memcmp` comparisons exercised by the oracle. It does not individually validate the parameter and return conventions of the other replaced routines or demonstrate removal of artifacts from the address space.

2) Operational Outcomes in the Corpus

The evaluation used 100 Windows PE executables, comprising 50 benign and 50 malicious samples. Each execution profile included 25 benign and 25 malicious executables. Ten attempts per binary and condition yielded 1,000 baseline attempts and 1,000 with `InstMemcmpMask`. The comparison used `-q` and `-dm -q`, without explicitly specifying the `-sf` option in the condition parameters. Order was randomized within blocks defined by binary and repetition.

The collector did not record the exit code in 777 baseline attempts and 780 attempts with the countermeasure. These records do not distinguish normal termination from failure. All these attempts remain in Table 13, without being classified as successful completions or confirmed failures.

Among persistent executables, the difference in observation-window attainment was -1.0 percentage point, with a 95% confidence interval (CI) of [-3.2, +1.2]. This exploratory interval was computed using a paired bootstrap over the 50 binaries, stratified by class, with 2,000 resamples and seed 705851366. The interval includes zero and does not establish equivalence between conditions. Process survival also does not demonstrate that the same tasks were performed.

3) Observed Duration and Resource Utilization

Table 14 presents the medians of the differences computed within each binary from the medians recorded in each condition. The 95% CIs were computed using the percentile method, with 2,000 paired resamples of binaries, seed 705851366, and reinitialization of the generator for each stratum and metric.

The filters for this reanalysis were applied post hoc: finite, positive duration in both conditions; finite CPU and memory values and at least one resource observation per condition. A total of 98 pairs were retained. Executables `MAL_33` and `MAL_37` had zero recorded duration and no resource measurements in either condition; they were excluded only from these contrasts, while their attempts remained in the operational-outcome table.

All duration intervals included zero. In the malicious short-lived stratum, the median peak-memory difference was +0.451 MB, with a 95% CI of [0.049, 0.805] MB. The other memory intervals included zero. These results are exploratory and do not demonstrate behavioral equivalence or absence of cost.

CPU-time summaries had a median of zero in both conditions across all strata. However, 19 of the 98 binaries with available paired records had a positive median CPU time in the baseline, compared with 20 with masking enabled. In addition, 21 pairs had nonzero CPU-time differences. The supplied summaries do not allow the origin of the numerous zero values to be verified; therefore, this metric does not support a conclusive estimate of CPU cost.

The coverage of `InstMemcmpMask` depends on the use of a replaced routine, the signature's position in the second operand, and an exact match to the expected length. Scans implemented directly by the program, comparisons with reversed operands, partial searches, unmonitored functions, and inspections of memory-page properties remain outside this scope.

Internal counters remained disabled in the corpus campaign, and the frequency of calls to the replaced routines was not verified. The operational and resource results do not establish which executables searched for the monitored signatures or whether masking allowed additional malicious activity to be observed.

E. SKEWMASK

Dynamic translation and the execution of analysis routines introduce delays that may alter the time perceived by the instrumented process [6, 9, 10, 11]. Evasive applications may measure the interval between two execution points, compare initial JIT-translated executions with subsequent cached executions, or correlate readings from different time sources with an expected threshold [13, 22].

The `SkewMask` countermeasure, enabled by `-do`, replaces `Sleep`, `SleepEx`, and `NtDelayExecution`. For each monitored relative wait, the module measures the observed interval using `QueryPerformanceCounter`. The observed and requested durations are expressed in the same time base before calculating the excess:

TABLE 13. Recorded InstMemcmpMask outcomes by execution profile. Each profile contains 50 executables and 500 attempts per condition.

Recorded outcome	Modules disabled `-q`	Masking enabled `-dm -q`	Difference (pp)
(a) Short-lived executables			
Termination with unavailable exit code	406 (81.2%)	404 (80.8%)	−0.4
120-s timeout	88 (17.6%)	92 (18.4%)	+0.8
Launch failure	6 (1.2%)	4 (0.8%)	−0.4
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**
(b) Persistent executables			
Termination with unavailable exit code	371 (74.2%)	376 (75.2%)	+1.0
Observation-window attainment at 60 s	106 (21.2%)	101 (20.2%)	−1.0
Launch failure	23 (4.6%)	23 (4.6%)	0.0
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**

Cells report counts (percentages of 500 attempts). Difference: the masking-enabled percentage minus the baseline percentage, in percentage points (pp). All attempts are included.

TABLE 14. Paired differences in observed duration and peak memory with InstMemcmpMask. Estimates are medians of per-binary differences, enabled condition minus baseline; 95% CIs are shown in brackets.

Class and profile	n	Duration difference (ms)	Peak-memory difference (MB)
Benign, short-lived	25	−36.2 [−183.0, 165.2]	−0.080 [−0.641, 0.604]
Benign, persistent	25	−6.3 [−108.4, 207.7]	+0.244 [−0.121, 0.445]
Malicious, short-lived	25	−19.9 [−283.9, 121.0]	+0.451 [0.049, 0.805]
Malicious, persistent	23	−71.9 [−538.6, 17.2]	−0.422 [−2.148, 0.504]

n denotes paired binaries with available measurements. MB retains the unit identified in the supplied records. The contrasts describe the observed executions without verification that equivalent work was completed.

$$\delta_s = \max\left(0, \Delta t_{\text{obs}} - \Delta t_{\text{req}}\right), \tag{1}$$

where Δt_{obs} denotes the observed interval and Δt_{req} denotes the requested wait. The maximum operation prevents a negative correction when the observed interval does not exceed the requested wait. The amount corresponding to the excess is expressed in performance-counter units and accumulated during execution.

The accumulated offset is used to adjust the values returned by `QueryPerformanceCounter`, `RtlQueryPerformanceCounter`, `GetTickCount`, `GetTickCount64`, `GetSystemTimePreciseAsFileTime`, and `QueryInterruptTimePrecise`. The objective is to compensate for the excess observed during monitored waits. Applying an accumulated offset, by itself, does not establish consistency among all these time sources.

1) Controlled Functional Validation

The `TestOverhead.exe` oracle queries `GetSystemTimePreciseAsFileTime` before and after a 50-ms call to `Sleep`. The observed duration in milliseconds is multiplied by 40, yielding the score $S = 40\,\Delta t_{\text{ms}}$. The threshold corresponds to 1.5 times the scaled requested wait:

$$T = 1.5 \times (50 \times 40) = 3\,000. \tag{2}$$

The controlled test used a single test program, separate from the 100 executables in the corpus. The number of attempts was fixed at 30 per condition, and all 90 records from the three conditions were retained. The conditions compared were `-q`, `-go -q` and `-go -do -q`. The option `-go` introduces the artificial test load and is retained in both conditions of the main comparison; thus, its removal is not confounded with enabling `-do`. The condition `-q` represents Pin running TOMWare.M with its modules disabled and no artificial load.

Table 15 presents the recalculated distributions of individual scores. For the 30 observations, the median is the average

of the two central values; quartiles use linear interpolation. The exploratory 95% confidence intervals (CIs) for the medians use 2,000 bootstrap resamples of the attempts, with seed 705851366 reset for each condition. The proportion intervals use the Wilson method. These intervals assume independence between attempts and do not model possible temporal dependence.

With the artificial load retained, the proportion of scores below the threshold increased from 11/30 to 30/30. This result supports the compensation observed in this specific test without establishing a universal rate of instrumentation concealment. The instrumented condition without artificial load also yielded 30/30 scores below the threshold; therefore, the oracle did not identify Pin in that condition.

In the wait test, only the native run produced the expected measurements. Both instrumented conditions timed out, including the baseline with artificial load. Thus, the result neither allows the timeout to be attributed exclusively to the countermeasure nor validates monotonicity, concurrent waits, or alertable-wait behavior under Pin. Accumulating excess durations during overlapping waits requires verifying that shared intervals are not subtracted more than once; the available data do not establish this property.

2) Operational Outcomes in the Corpus

The corpus evaluation used 100 Windows PE executables: 50 benign and 50 malicious. Each class included 25 short-lived and 25 persistent executables. Ten attempts per binary and condition yielded 1,000 attempts in the baseline and 1,000 with `SkewMask`. The comparison used `-q` and `-do -q`, without `-go`, with randomized order within blocks defined by binary and repetition. This experiment is separate from the 90 attempts with the timing oracle.

Across the 1,000 attempts in each condition, the baseline recorded 777 terminations with unavailable exit codes, 88 timeouts, 106 instances of observation-window attainment, and 29 launch failures. With SkewMask, these totals were 580, 120, 276, and 24, respectively. Missing exit codes prevent normal terminations from being distinguished from failures. All attempts remain in Table 16, without being classified as successful completions or confirmed failures.

Among persistent executables, the difference in observation-window attainment was +34.0 percentage points, with a 95% CI of [24.0, 44.0]. This exploratory interval was calculated using a paired bootstrap over the 50 binaries, stratified by class, with 2,000 resamples and seed 705851366. Greater observation-window attainment does not establish greater execution progress: a process may reach the observation window while waiting or performing different operations.

3) Observed Duration and Resource Utilization

Table 17 presents the medians of duration and memory differences calculated within each binary. The 95% CIs use the percentile method, with 2,000 paired resamples of binaries, seed 705851366, and the generator reset for each stratum and metric. The CPU columns provide only descriptive summaries of the values recorded in each condition.

The filters in this reanalysis were applied post hoc: finite, positive durations in both conditions; finite CPU and memory values and at least one resource observation in each condition. A total of 98 pairs were retained. The executables `MAL_33` and `MAL_37` recorded zero duration and no resource measurements in either condition; they were excluded only from these contrasts, with their attempts retained in the operational outcomes.

All four strata showed positive duration differences, with CIs entirely above zero. The increases in observation-window attainment and duration are consistent with changes in execution state or pace, but the records do not identify the programs' behavioral progress. In the persistent malicious stratum, the observed reduction in peak memory is not interpreted as an efficiency gain, because the execution of equivalent work was not demonstrated.

Among the 98 pairs, 19 binaries had positive median CPU times in the baseline, compared with 48 with `SkewMask`; 49 pairs showed nonzero CPU-time differences. In the persistent strata, the recorded values were higher with the countermeasure. However, the origin of the numerous zero values in the baseline cannot be validated from the supplied summaries. Therefore, the CPU values are descriptive and do not support a conclusive estimate of the cost of compensation.

The coverage of `SkewMask` does not include direct readings of `RDTSC` or `RDTSCP`, unmonitored time sources, external counters, or absolute waits supplied to `NtDelayExecution`. The threshold-test results are limited to the combination of timing operations exercised by `Sleep` and `GetSystemTimePreciseAsFileTime`, without establishing consistency across all replaced sources.

In the corpus, internal counters remained disabled, and the timing checks performed by each executable were not identified. The results do not demonstrate the recovery of suppressed malicious behavior or the preservation of system calls, control flow, or external effects.

VI. EXPERIMENTAL EVALUATION AND RESULTS

The evaluation separates indicator suppression in controlled tests from operational outcomes and resource measurements in the corpus. Functional tests assess specific interfaces; the corpus evaluation does not assume that each executable performs these checks.

A. EXPERIMENTAL ENVIRONMENT

The experiments used a host running Windows 11 Home x64 with 16 GB of RAM and VMware Workstation Pro 25.0.1. The virtual machine ran Windows 10 x64 with 8 GB of RAM, four vCPUs, and a 60-GB virtual NVMe disk. The network operated in host-only mode. Instrumentation used Intel Pin 3.28 and TOMWare.M compiled for x64.

TABLE 15. SkewMask functional tests, separate from the evaluation of the 100-executable corpus. Panel (a): 30 repetitions per condition, totaling 90 attempts with the timing oracle. Panel (b): one run per condition in the wait test.

(a) TestOverhead.exe: 30 repetitions per condition

Condition	Median [95% CI]	[Q1; Q3]	[Min.; Max.]	Scores below T n (%) [95% CI]
No artificial load `-q`	2 433.29 [2 326.33, 2 523.36]	[2 262.90, 2 560.13]	[2 168.12, 2 749.98]	30/30 (100.0%) [88.6, 100.0]
Artificial load `-go -q`	3 083.21 [2 997.01, 3 171.09]	[2 985.30, 3 189.17]	[2 722.01, 3 845.31]	11/30 (36.7%) [21.9, 54.5]
Load and SkewMask `-go -do -q`	2 104.77 [2 096.06, 2 108.87]	[2 091.35, 2 112.75]	[2 073.28, 2 146.11]	30/30 (100.0%) [88.6, 100.0]

(b) TestConcurrentWaits.exe: one run per condition

Condition	Sequential (ms)	Concurrent (ms)	Reported monotonicity	Alertable wait (ms)
Native, without Pin	413	203	Yes	31
`-go -q`	Timeout, no timing measurements available			
`-go -do -q`	Timeout, no timing measurements available			

Scores below the threshold: $S < T$. Scores are durations in milliseconds multiplied by 40, not CPU cycles. Proportion CIs are expressed as percentages. In the wait test, the native record also reports a 62-ms interval between time readings.

TABLE 16. SkewMask outcomes in the 100-executable corpus, with 1,000 attempts per condition. Each profile comprises 50 executables and 500 attempts per condition.

Recorded outcome	Modules disabled `-q`	SkewMask enabled `-do -q`	Difference (pp)
(a) Short-lived: 50 binaries; 500 attempts per condition			
Termination with unavailable exit code	406 (81.2%)	377 (75.4%)	−5.8
120-s timeout	88 (17.6%)	120 (24.0%)	+6.4
Launch failure	6 (1.2%)	3 (0.6%)	−0.6
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**
(b) Persistent: 50 binaries; 500 attempts per condition			
Termination with unavailable exit code	371 (74.2%)	203 (40.6%)	−33.6
Observation-window attainment at 60 s	106 (21.2%)	276 (55.2%)	+34.0
Launch failure	23 (4.6%)	21 (4.2%)	−0.4
Profile total	**500 (100.0%)**	**500 (100.0%)**	**0.0**

Cells: count (percentage of 500 attempts). Difference: percentage with SkewMask minus the baseline percentage, in percentage points (pp). The two profiles total 100 binaries and 1,000 attempts per condition; no attempts were excluded from this table.

B. CORPUS AND EXPERIMENTAL DESIGN

The corpus contains 100 PE x64 executables, balanced between 50 benign and 50 malicious executables, with 25 binaries of each profile per class. The manifest identifies files by SHA-256 and records their source, classification, architecture, packing, and execution profile. The declared sources are the Windows environment for benign executables and MalwareBazaar for malicious executables. This balance does not establish representativeness with respect to the prevalence of evasion.

Nine conditions were evaluated: native execution (NAT), Pin with an empty Pintool (EMPTY), TOMWare.M with modules disabled (TMW0), each of the five countermeasures individually, and joint activation of all modules. The condition using Pin without a Pintool was not included in this campaign following an unsuccessful preliminary check; therefore, EMPTY does not separate the engine cost from the cost associated with loading the tool. The incremental effect of each countermeasure was evaluated relative to TMW0.

Ten attempts per binary and condition yielded

$$N = 100 \times 9 \times 10 = 9\,000. \tag{3}$$

The order of conditions was randomized within each block defined by binary and repetition, using seed 705851366. The clean snapshot was restored for each block. The profiles

TABLE 17. SkewMask duration and resources: 98 eligible pairs among the 100 evaluated executables. Paired duration and memory differences with 95% CIs; CPU values are presented descriptively.

Class and profile	n	Duration difference (s)	CPU time (s)		Peak memory difference (MB)
			Baseline	SkewMask	
Benign, short-lived	25	+31.669 [13.755, 51.072]	0.000	0.000	+0.709 [−0.191, 2.477]
Benign, persistent	25	+55.452 [30.685, 57.031]	0.000	24.352	−3.174 [−58.299, 0.355]
Malicious, short-lived	25	+34.183 [10.571, 62.294]	0.000	0.000	+1.631 [0.811, 2.336]
Malicious, persistent	23	+37.387 [26.145, 45.375]	0.000	23.828	−22.678 [−146.469, −7.904]

n denotes binaries with eligible paired measurements, totaling 98. The other two binaries remain in the corpus outcomes. Differences: the baseline median is first subtracted from the median with the countermeasure enabled for each binary; the median of these differences is then calculated. 95% CIs are shown in brackets. CPU: median of the medians recorded per binary; zero does not imply an absence of CPU consumption. MB retains the unit identified in the supplied records.

TABLE 18. Updated corpus composition.

Class	Short-lived	Persistent	Total
Benign	25	25	50
Malicious	25	25	50
Total	**50**	**50**	**100**

recorded in the manifest determined the 120-s timeout for short-lived executables and the 60-s window for persistent executables, with resource sampling every 200 ms. Classification as persistent does not mean that all attempts reached the window. Failures and timeouts were retained in the outcome analysis.

C. FUNCTIONAL VALIDATION

The five oracles assessed debugging, enumeration, the environment, memory comparisons, and timing. Their results, presented in the countermeasure subsections, were not counted as evidence of evasion in the corpus binaries. The evidence for ProcessEnum, SanitizePinEnvVars, and InstMemcmpMask is limited to the interfaces and signatures exercised; in particular, the absence of monitored modules in the baseline does not provide a positive control for their concealment.

AntiDebug was evaluated with the same controlled stimulus in the conditions `-gdb -q` and `-gdb -dd -q`. The monitored PEB indicators changed from positive in 3/3 repetitions to absent in 3/3, without demonstrating the concealment of a real debugger.

The SkewMask test fixed 30 repetitions in each of three conditions, retaining all 90 attempts. With the artificial load retained, scores below the threshold increased from 11/30 with `-go -q` to 30/30 with `-go -do -q`. The condition without artificial load also yielded 30/30 scores below the threshold. These results are specific to the oracle and do not constitute an evasion rate in the corpus. Both instrumented conditions in the concurrent-wait test timed out; clock consistency under Pin remains unvalidated.

D. OPERATIONAL OUTCOMES IN THE CORPUS

Table 19 includes all 9,000 attempts. The collector's previous label, `crash_exit`, was corrected to termination with unavailable exit code. These records establish neither failure nor successful completion; therefore, no success rate is calculated from them.

Among persistent executables, observation-window attainment at 60 s increased from 21.2% with TMW0 to 55.2% with SkewMask and 60.2% with all modules. For SkewMask, the paired difference was +34.0 percentage points, with a 95% CI of [24.0, 44.0]. However, timeouts among short-lived executables increased from 17.6% to 24.0%. Greater observation-window attainment does not demonstrate equivalent progress or the recovery of malicious behavior.

E. DURATION, RESOURCES, AND ANALYSIS CRITERIA

Repetitions were summarized by the median for each binary and condition. For a metric M, the contrast was calculated within each binary before aggregation by class and profile:

$$d_i(M) = \widetilde{M}_{i,\mathrm{CM}} - \widetilde{M}_{i,\mathrm{TMW0}},$$
$$\widehat{\Delta}(M) = \mathrm{median}_i\, d_i(M). \tag{4}$$

The exploratory 95% CIs used 2,000 bootstrap resamples of paired binaries, seed 705851366, and percentiles with linear interpolation. For resources, the generator was reset for each stratum and metric. For observation-window attainment, the mean difference in per-binary rates was estimated using paired resampling stratified by class. Attempts from the same binary were not treated as independent samples in this analysis.

The filters were defined post hoc: finite, positive durations in both conditions; finite CPU and memory values and at least

TABLE 19. Outcomes across the nine conditions. Each condition comprises 100 binaries and 1,000 attempts, divided into 500 per profile.

	Short-lived: 50 binaries			Persistent: 50 binaries		
Condition	No exit code	Timeout	Launch fail.	No exit code	60-s window	Launch fail.
NAT	403 (80.6%)	88 (17.6%)	9 (1.8%)	314 (62.8%)	156 (31.2%)	30 (6.0%)
EMPTY	404 (80.8%)	94 (18.8%)	2 (0.4%)	368 (73.6%)	109 (21.8%)	23 (4.6%)
TMW0	406 (81.2%)	88 (17.6%)	6 (1.2%)	371 (74.2%)	106 (21.2%)	23 (4.6%)
TMW_DD	406 (81.2%)	89 (17.8%)	5 (1.0%)	377 (75.4%)	99 (19.8%)	24 (4.8%)
TMW_DP	405 (81.0%)	91 (18.2%)	4 (0.8%)	380 (76.0%)	98 (19.6%)	22 (4.4%)
TMW_DE	409 (81.8%)	90 (18.0%)	1 (0.2%)	368 (73.6%)	109 (21.8%)	23 (4.6%)
TMW_DM	404 (80.8%)	92 (18.4%)	4 (0.8%)	376 (75.2%)	101 (20.2%)	23 (4.6%)
TMW_DO	377 (75.4%)	120 (24.0%)	3 (0.6%)	203 (40.6%)	276 (55.2%)	21 (4.2%)
TMW_ALL	369 (73.8%)	125 (25.0%)	6 (1.2%)	177 (35.4%)	301 (60.2%)	22 (4.4%)

Cells: count (percentage of the 500 attempts in the profile). No exit code: termination with unavailable exit code. Timeout: 120-s limit. Launch fail.: launch failure. NAT: native execution; EMPTY: Pin with an empty Pintool; TMW0: modules disabled. DD, DP, DE, DM, and DO correspond to AntiDebug, ProcessEnum, SanitizePinEnvVars, InstMemcmpMask, and SkewMask; ALL enables all five modules.

one resource observation in each condition. There were 98 eligible pairs per countermeasure for duration. CPU and memory also had 98 pairs, except for SanitizePinEnvVars, which had 97. `MAL_33` and `MAL_37` did not provide eligible measurements; for SanitizePinEnvVars, `BEN_32` also did not provide CPU and memory measurements. Their attempts were retained in the operational outcomes.

Table 20 shows the largest duration increases with ProcessEnum and SkewMask in both short-lived groups. The memory reduction with SkewMask among persistent malicious executables does not demonstrate an efficiency gain, because equivalent work was not established. CPU time is presented only descriptively in the per-module analyses: the origin of the numerous zero values could not be validated from the available summaries. A recorded zero does not imply an absence of consumption, and cost percentages based on these values are not interpreted.

F. SUMMARY OF RESEARCH QUESTIONS

RQ1. The tests support the suppression of the exercised indicators within each module's coverage. The timing result pertains to the oracle with a controlled load; clock consistency during concurrent waits remains unvalidated.

RQ2. The campaign characterizes outcomes, duration, and resources under the evaluated conditions. Internal counters were disabled, and the evasion mechanisms triggered by each binary were not verified. Thus, the results do not establish behavioral fidelity, coverage of descendant processes, or the recovery of suppressed malicious behavior.

VII. DISCUSSION

A. FUNCTIONAL EFFECTIVENESS OF THE COUNTERMEASURES

The evidence for RQ1 supports the suppression of indicators exercised by the oracles, within the scope of the checks performed. `SanitizePinEnvVars` renamed the monitored variable, preventing its lookup under the original name. `InstMemcmpMask` changed the results of intercepted comparisons while preserving the data in memory. `ProcessEnum` hid the monitored process entries; the absence of target modules in the baseline prevents confirmation of their filtering through Module32.

In the AntiDebug evaluation, the same controlled stimulus was retained in the conditions `-gdb -q` and `-gdb -dd -q`. The PEB indicators changed from positive in 3/3 repetitions to absent in 3/3. The result is limited to the fields `BeingDebugged` and `NtGlobalFlag`, under the simulated stimulus, without validating the concealment of a real debugger.

The SkewMask test used 30 repetitions per condition, retaining all 90 attempts. With the artificial load retained, scores below the threshold increased from 11/30 to 30/30. Since the condition without load also yielded 30/30 scores below the threshold, the result demonstrates the effect on this oracle without establishing a general rate of Pin concealment. Both instrumented conditions in the concurrent-wait test timed out, leaving temporal consistency in this scenario unvalidated.

The observed suppression depends on the queried interface and the detection strategy. Alternative queries, unmonitored signatures, or checks implemented directly by the program may yield different results [9, 10].

B. OPERATIONAL OUTCOMES AND BEHAVIORAL PRESERVATION

The campaign of 100 binaries and 9,000 attempts characterizes outcomes across the nine evaluated conditions. Terminations with unavailable exit codes prevent claims of full compatibility or successful completion of the applications. Launch failures and timeouts remain in the results and are not counted as operational successes.

Among persistent executables, observation-window attainment at 60 s was 21.2% with TMW0, 55.2% with SkewMask, and 60.2% with all modules. This greater observation-window attainment extends the interval during which the process can be observed, but may result from waiting, prolonged ini-

TABLE 20. Summary of incremental contrasts: median of per-binary differences, countermeasure minus TMW0, with 95% CIs in brackets.

	Observed duration (s) Short-lived		Peak memory (MB) Persistent	
Countermeasure	**Benign**	**Malicious**	**Benign**	**Malicious**
`AntiDebug`	−0.037 [−0.313, 0.107]	+0.149 [0.025, 0.842]	−0.008 [−0.533, 0.670]	−0.406 [−1.629, 0.482]
`ProcessEnum`	+3.844 [3.427, 4.088]	+2.784 [0.052, 4.083]	+0.434 [−0.855, 1.281]	+0.219 [−0.438, 0.680]
`SanitizePinEnvVars`	−0.007 [−0.179, 0.163]	+0.007 [−0.048, 0.461]	+0.488 [−0.688, 1.129]	+0.188 [−0.348, 0.574]
`InstMemcmpMask`	−0.036 [−0.183, 0.165]	−0.020 [−0.284, 0.121]	+0.244 [−0.121, 0.445]	−0.422 [−2.148, 0.504]
`SkewMask`	+31.669 [13.755, 51.072]	+34.183 [10.571, 62.294]	−3.174 [−58.299, 0.355]	−22.678 [−146.469, −7.904]

Duration: 25 pairs in each short-lived class. Memory: peak working set, with 25 benign pairs (24 for SanitizePinEnvVars) and 23 malicious pairs among persistent executables. Observed duration does not necessarily equal the time required to complete a task. MB retains the unit identified in the records.

tialization, or changes in the executed control flow. It does not, by itself, demonstrate more useful activity for the analyst.

Internal counters were disabled, and the evasion checks performed by each binary were not identified. The condition with all modules enabled also does not establish that every module was triggered or that their interactions preserved behavior. Demonstrating the recovery of suppressed behavior would require linking an identified check to its branch and to an additional observable effect, such as unpacked code, a recovered configuration, or file, registry, or network activity. This link was not established in this evaluation.

C. MODULARITY AND METRIC INTERPRETATION

Selective activation allows analysts to compare each module with TMW0 and select the countermeasures relevant to the experiment. However, a difference between configurations does not necessarily represent the internal cost of the hooks: changes in program progress and in the frequency of intercepted operations may also influence measured resource use [19].

In Table 20, ProcessEnum and SkewMask showed the largest increases in observed duration in the short-lived groups. For SkewMask, the median reduction of 22.678 MB in peak memory among persistent malicious executables does not demonstrate an efficiency gain, because equivalent work was not established. Altering perceived time is particularly sensitive because instrumentation itself modifies the application's timing measurements [6, 11, 22].

Confidence intervals quantify variation between binaries in the analyzed contrasts. An interval that includes zero does not establish equivalence between conditions. The origin of the numerous zero CPU values remains unvalidated in the available summaries; therefore, these measurements are descriptive and do not support conclusive estimates of cost or savings expressed as percentages.

No direct experimental comparison with Arancino was conducted [4]. Thus, the demonstrated modularity does not establish superiority in coverage, compatibility, or performance over this solution or other analysis environments.

D. LIMITATIONS AND THREATS TO VALIDITY

Coverage is limited to the structures, interfaces, signatures, and time sources described in Section V. Unintercepted native queries, custom comparison routines, residual copies of artifacts, and direct readings via `RDTSC` or `RDTSCP` remain outside this scope. The controlled oracles also do not represent the diversity or combinations of checks employed by protected and malicious programs.

NAT and EMPTY provide additional baselines for the incremental comparison with TMW0. However, the campaign did not include Pin without a Pintool; therefore, the cost attributable exclusively to the engine was not separated from the cost associated with loading the tool. In addition, TMW0 may exhibit behavior different from native execution.

The ten attempts per condition are clustered within each binary; the 9,000 attempts do not constitute 9,000 independent observations. The paired bootstrap at the binary level accounts for this clustering. Block randomization reduces systematic order effects, but restoring the snapshot only between blocks does not rule out possible carryover effects between conditions within the same block. Eligibility filters were defined post hoc, and missing measurements reduced the number of available pairs for some contrasts.

External validity remains limited to the balanced corpus, Windows 10 x64, Intel Pin 3.28, and the virtualized infrastructure used. The expansion to 100 binaries broadens the operational evaluation without guaranteeing representativeness of families, protectors, or evasion mechanisms. The absence of comparisons of API traces, code coverage, execution milestones, and external effects limits the conclusions for RQ2 to the recorded outcomes and metrics. Preservation of the behavior of descendant processes was also not demonstrated.

VIII. CONCLUSION AND FUTURE WORK

This work investigated five selectable countermeasures to mitigate instrumentation indicators in Windows executables, covering debugging states, process and module enumeration, environment variables, memory signatures, and timing skew. The evaluation examined the suppression of monitored indicators and the operational effects of applying these countermeasures under Intel Pin.

The oracles demonstrated suppression of environment, process enumeration, and memory comparison indicators for the interfaces and conditions exercised. In `AntiDebug`, the PEB indicators changed from positive in 3/3 repetitions to absent in 3/3, with the same simulated stimulus retained. This result remains limited to the two monitored fields.

The `SkewMask` test used a fixed number of 30 repetitions per condition and retained all 90 attempts. With the artificial load retained, scores below the threshold increased from 11/30 to 30/30. This result characterizes compensation in the specific oracle. Temporal consistency during concurrent waits remains unvalidated because both instrumented conditions in that test timed out.

The operational campaign evaluated 100 PE x64 executables, comprising 50 benign and 50 malicious executables, under nine conditions with ten attempts per binary and condition, totaling 9,000 attempts. The conditions included native execution, execution with an empty Pintool, TOMWare.M with modules disabled, individual activation of the five countermeasures, and joint activation of all modules. Contrasts were calculated per binary and accompanied by confidence intervals.

The results characterize differences in outcomes, observed duration, and recorded resources. Unavailable exit codes and limitations of the CPU measurements preclude claims of full compatibility or conclusive estimates of the cost of all modules. Greater observation-window attainment or lower memory consumption also does not establish equivalent work. The contribution is a platform for studying targeted countermeasures, without establishing universal transparency, behavioral preservation, or recovery of suppressed malicious behavior.

Future work will prioritize:

1) validating temporal consistency during sequential, concurrent, and alertable waits, while expanding coverage of time sources;
2) completing positive controls for supported interfaces and further evaluating interactions between modules;
3) improving the collection of exit codes and CPU measurements and comparing execution milestones, descendant processes, API traces, code coverage, and external effects;
4) demonstrating practical benefits in documented evasive cases by linking the detected check to the additional behavior observed; and
5) diversifying the corpus and comparing defensive solutions under equivalent experimental conditions.

REFERENCES

[1] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, "Pin: Building customized program analysis tools with dynamic instrumentation," in *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '05. New York, NY, USA: Association for Computing Machinery, 2005, pp. 190–200.

[2] R. J. Rodríguez, I. Rodríguez-Gastón, and J. Alonso, "Towards the detection of isolation-aware malware," *IEEE Latin America Transactions*, vol. 14, no. 2, pp. 1024–1036, Feb. 2016.

[3] A. Santos Filho, R. J. Rodríguez, and E. L. Feitosa, "Evasion and countermeasures techniques to detect dynamic binary instrumentation frameworks," *Digital Threats: Research and Practice*, vol. 3, no. 2, pp. 1–28, Feb. 2022, art. no. 11.

[4] M. Polino, A. Continella, S. Mariani, S. D'Alessio, L. Fontana, F. Gritti, and S. Zanero, "Measuring and defeating anti-instrumentation-equipped malware," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, ser. Lecture Notes in Computer Science, vol. 10327. Cham, Switzerland: Springer, 2017, pp. 73–96.

[5] M. Kim, H. Cho, and J. H. Yi, "Large-scale analysis on anti-analysis techniques in real-world malware," *IEEE Access*, vol. 10, pp. 75 802–75 815, 2022.

[6] S. Park and Y. Park, "Detection techniques for DBI environment in Windows," *Electronics*, vol. 13, no. 5, 2024, art. no. 871.

[7] Y. B. Lee, J. H. Suk, and D. H. Lee, "Bypassing anti-analysis of commercial protector methods using DBI tools," *IEEE Access*, vol. 9, pp. 7655–7673, 2021.

[8] J. Park, Y.-H. Jang, S. Hong, and Y. Park, "Automatic detection and bypassing of anti-debugging techniques for Microsoft Windows environments," *Advances in Electrical and Computer Engineering*, vol. 19, no. 2, pp. 23–28, 2019.

[9] D. C. D'Elia, E. Coppa, S. Nicchi, F. Palmaro, and L. Cavallaro, "SoK: Using dynamic binary instrumentation for security (and how you may get caught red handed)," in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, ser. Asia CCS '19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 15–27.

[10] D. C. D'Elia, L. Invidia, F. Palmaro, and L. Querzoni, "Evaluating dynamic binary instrumentation systems for conspicuous features and artifacts," *Digital Threats: Research and Practice*, vol. 3, no. 2, pp. 1–13, Feb. 2022, art. no. 10.

[11] D. Bruening, Q. Zhao, and S. Amarasinghe, "Transparent dynamic instrumentation," in *Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments*, ser. VEE '12. New York, NY, USA:

Association for Computing Machinery, 2012, pp. 133–144.

[12] F. Falcón and N. Riva, "Dynamic binary instrumentation frameworks: I know you're there spying on me," Presented at REcon 2012, 2012, accessed: Jul. 30, 2026. [Online]. Available: https://recon.cx/2012/schedule/events/216.en.html

[13] J. Kirsch, Z. Zhechev, B. Bierbaumer, and T. Kittel, "PwIN: Pwning Intel Pin: Why DBI is unsuitable for security applications," in *Computer Security – ESORICS 2018*, ser. Lecture Notes in Computer Science, vol. 11098. Cham, Switzerland: Springer, 2018, pp. 363–382.

[14] F. Palmaro and L. Franchina, "Beware of unknown areas to notify adversaries: Detecting dynamic binary instrumentation runtimes with low-level memory scanning," in *Intelligent Computing*, ser. Lecture Notes in Networks and Systems, vol. 285. Cham, Switzerland: Springer, 2021, pp. 1003–1019.

[15] S. T. Padhy, P. B. Shanthi, G. Bijur, and S. Hemalatha, "A threat-informed approach to malware evasion using DRM and TLS callbacks," *IEEE Access*, vol. 13, pp. 155 195–155 202, 2025.

[16] A. Santos Filho, R. J. Rodríguez, and E. L. Feitosa, "Reducing the attack surface of dynamic binary instrumentation frameworks," in *Developments and Advances in Defense and Security*. Singapore: Springer, 2020, vol. 152, pp. 3–13.

[17] V. Crăciun, A. Mogage, and D. Lucanu, "Full transparency in DBI frameworks," 2023, arXiv:2306.13529. [Online]. Available: https://arxiv.org/abs/2306.13529

[18] M. Gaber, M. Ahmed, and H. Janicke, "Defeating evasive malware with Peekaboo: Extracting authentic malware behavior with dynamic binary instrumentation," *Journal of Information Security and Applications*, vol. 95, 2025, art. no. 104290.

[19] A. Norby, B. P. Rimal, and B. Brizendine, "Measurement of anti-debugging techniques on the Windows and Linux operating systems for the Intel x86_64 architecture," *IEEE Access*, vol. 13, pp. 46 568–46 583, 2025.

[20] J. Zeng, Y. Fu, and Z. Lin, "PEMU: A Pin highly compatible out-of-VM dynamic binary instrumentation framework," in *Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE '15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 147–160.

[21] C. Zhao, Y. Beugin, J.-C. Noirot Ferrand, Q. Burke, G. Li, and P. McDaniel, "LibIHT: A hardware-based approach to efficient and evasion-resistant dynamic binary analysis," in *Proceedings of the 2025 Workshop on Software Understanding and Reverse Engineering*. New York, NY, USA: Association for Computing Machinery, 2025, pp. 89–101.

[22] Y. Oyama, "How does malware use RDTSC? A study on operations executed by malware with CPU cycle measurement," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, ser. Lecture Notes in Computer Science, vol. 11543. Cham, Switzerland: Springer, 2019, pp. 197–218.

[23] H. Shi and J. Mirkovic, "Hiding debuggers from malware with Apate," in *Proceedings of the 32nd Annual ACM Symposium on Applied Computing*, ser. SAC '17. New York, NY, USA: Association for Computing Machinery, 2017, pp. 1703–1710.

[24] F. Zhang, K. Leach, A. Stavrou, and H. Wang, "Towards transparent debugging," *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 2, pp. 321–335, 2018.



FRANCISCO S. S. NETO Francisco Soares de Sousa Neto received the B.Sc. degree in Information Systems from Faculdade Metropolitana de Manaus, Manaus, Brazil, in 2011, and the M.Sc. degree in Informatics from the Federal University of Amazonas (UFAM), Manaus, Brazil, in 2017. He is currently pursuing the Ph.D. degree in Computer Science with the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Manaus, Brazil. His research interests include cybersecurity, malware analysis, dynamic binary instrumentation, anti-analysis techniques, and software protection.



MARGEFSON M. BARROS Margefson Machado Barros received the B.Sc. degree in Information Systems from Universidade Nilton Lins, Manaus, Brazil, in 2009, and the postgraduate degree in Web Application Development from FUICAPI. He is currently pursuing a postgraduate degree in Cybersecurity at the Federal University of Amazonas (UFAM), Manaus, Brazil. He works as a Cybersecurity Resident Analyst at Lenovo, CSOC team. His research interests include cybersecurity, malware analysis, dynamic binary instrumentation, anti-analysis techniques, software protection, and security operations.



EDUARDO L. FEITOSA Eduardo Luzeiro Feitosa received the B.Sc. degree in Data Processing from the Federal University of Amazonas (UFAM), Manaus, Brazil, in 1998, the M.Sc. degree in Computer Science from the Federal University of Rio Grande do Sul, Porto Alegre, Brazil, in 2001, and the Ph.D. degree in Computer Science from the Federal University of Pernambuco, Recife, Brazil, in 2010. He is currently an Associate Professor with the Institute of Computing (IComp), Federal University of Amazonas (UFAM), Manaus, Brazil, and a permanent faculty member of the Graduate Program in Informatics (PPGI/UFAM). His research interests include cybersecurity, malware analysis, computer networks, and systems security.